KALYANI GOVERNMENT ENGINEERING COLLEGE

DEPARTMENT OF COMPUTER APPLICATION



PROJECT REPORT

ON

DESIGN A SIMULATOR FOR DETECTION OF OUTLIERS

OUTLIER HUNT

An Interactive Educational Simulator for Categorical and Statistical Outlier Detection

Submitted by

Student Name	Roll No.
Saswati Majumder	10271025034
Prerna Kumari Sharma	10271025022
Ankita Baital	10271025005

Programme: Master of Computer Applications (MCA)

Subject: Basic Data Science

Subject Code: MCANE304F

Subject Guide: Dr. Anuradha Banerjee

Kalyani, West Bengal

DECLARATION

We hereby declare that the project entitled “Design a Simulator for Detection of Outliers”, presented in this report under the title “Outlier Hunt”, has been prepared by us as an academic project for the subject Basic Data Science (MCANE304F) in the Master of Computer Applications programme at Kalyani Government Engineering College.

The report describes the design and implementation evidenced by the project source code, project presentation, and application screenshots. Statistical methods discussed in the theoretical chapters are distinguished from the features implemented in the simulator. No experimental outcomes, user-study findings, or performance measurements have been fabricated.

We further declare that sources consulted for theoretical and technical explanations are acknowledged in the references.

Saswati Majumder (Roll No. 10271025034)

Prerna Kumari Sharma (Roll No. 10271025022)

Ankita Baital (Roll No. 10271025005)

ACKNOWLEDGEMENT

The project team expresses its sincere gratitude to Dr. Anuradha Banerjee, Subject Guide for Basic Data Science, for academic guidance and support during the development of this project. The team also acknowledges the Department of Computer Application, Kalyani Government Engineering College, for providing the academic setting in which the work was undertaken.

The team is grateful to the contributors of the open-source technologies and documentation used in the implementation, particularly the React ecosystem and the supporting libraries identified in the project configuration. The project presentation, source repository, and application screenshots provided the principal evidence used in preparing this report.

ABSTRACT

Outlier detection is an important task in exploratory data analysis because unusual observations may indicate measurement errors, rare events, data-entry problems, or meaningful departures from an expected pattern. Students often encounter the topic through formulas and static examples, while the practical process of inspecting a dataset, selecting a detection rule, and interpreting a result receives less interactive treatment. This project presents Outlier Hunt, a browser-based educational simulator designed to support introductory learning through short game sessions.

The application separates conceptual odd-one-out exercises from numerical anomaly recognition and statistical testing. It provides four modes—Quick Hunt, Category Clash, Number Detective, and Data Scientist Challenge—and four difficulty levels. In the statistical mode, a learner can inspect a numerical dataset, identify a target observation, choose either the 1.5×IQR fence rule or a Z-score threshold, and make a binary judgement. The implementation computes descriptive statistics, quartiles, interquartile range, fences, and Z-scores, and displays a visual representation of the dataset. Game mechanics include timed questions, hints, score multipliers, streaks, immediate feedback, answer explanations, and a local performance history.

The system is implemented using React and TypeScript within a Vite/TanStack application structure. The question bank contains 48 curated questions, with 12 assigned to each mode; a five-question session is selected according to mode and difficulty, with a fallback to the mode pool where necessary. Session statistics are persisted in browser localStorage rather than a remote database. The project is an educational integration of established detection methods, not a proposal of a new statistical algorithm. Its contribution lies in combining conceptual classification, numerical pattern recognition, formula-based detection, interactive visualisation, and feedback in one learning interface.

Keywords: outlier detection, interquartile range, IQR, Z-score, educational simulator, React, data visualisation, game-based learning.

TABLE OF CONTENTS

1. Introduction

2. Problem Definition, Aim and Objectives

3. Theoretical Background

4. System Analysis and Requirements

5. System Design and Architecture

6. Implementation Details

7. User Interface and Screenshots

8. Testing and Verification

9. Novelty, Evaluation and Limitations

10. Conclusion and Future Scope

References

Appendix A: Important Formulae and Algorithms

LIST OF FIGURES

Figure 4.1. Game-mode and difficulty selection interface.

Figure 5.1. Data Scientist Challenge with dataset, target observation, descriptive statistics and IQR fences.

Figure 6.1. Personal Statistics panel showing locally stored performance history.

LIST OF TABLES

Table 2.1. Project objectives and corresponding system capabilities.

Table 4.1. Functional requirements.

Table 4.2. Non-functional requirements.

Table 5.1. Major software modules and responsibilities.

Table 6.1. Technologies and their roles.

Table 8.1. Verification cases derived from source-level tests and implemented behaviour.

Table 9.1. Comparison with common learning approaches.

LIST OF ABBREVIATIONS

Abbreviation	**Meaning**
IQR	Interquartile Range
Q1	First Quartile (25th percentile)
Q3	Third Quartile (75th percentile)
SD / σ	Standard Deviation
UI	User Interface
UX	User Experience
SPA	Single-Page Application
DOM	Document Object Model
JSON	JavaScript Object Notation

CHAPTER 1: INTRODUCTION

1.1 Background

A dataset is rarely composed of observations that are perfectly uniform. Even when most values follow a recognisable pattern, some observations may lie unusually far from the main body of data. Such observations are commonly called outliers. Their presence matters because they can affect summary statistics, distort visualisations, influence fitted models, and sometimes reveal important events. However, an unusual value is not automatically erroneous: it may represent a valid but rare event, a change in operating conditions, or a data-quality issue requiring investigation.

Outlier detection therefore involves more than identifying the largest or smallest number. It requires a reference for what is typical in a dataset, a rule for deciding what is sufficiently unusual, and an interpretation of the result in context. Common introductory approaches include the interquartile range rule and standardised scores. Visual inspection is also important because numerical rules depend on assumptions and may behave differently for skewed distributions, small samples, or influential extreme values.

1.2 Project Overview

Outlier Hunt is an interactive web application that teaches the idea of an outlier through four complementary modes. Two modes focus on intuitive or conceptual recognition, one focuses on numerical sequences, and one provides a statistical laboratory in which a learner can modify the dataset and detection rule. The interface uses a game-session structure to encourage repeated practice while displaying explanations and performance information.

1.3 Scope

The project scope is limited to an educational browser-based simulator for introductory outlier concepts. It includes a predefined question bank, generated practice datasets, descriptive-statistics calculations, IQR and Z-score classification, graphical display of observations and thresholds, game scoring, and browser-local performance history. It does not claim to provide a

production-grade anomaly-detection service, a machine-learning model, a multi-user account system, or a server-side analytics platform.

1.4 Organisation of the Report

The report proceeds from the problem and objectives to the statistical background, requirements, architecture, implementation, interface evidence, testing, novelty and limitations, and final conclusions. This order connects the educational motivation to the actual software design and the mathematical procedures implemented.

CHAPTER 2: PROBLEM DEFINITION, AIM AND OBJECTIVES

2.1 Problem Statement

Learners may understand the definition of an outlier yet find it difficult to apply a detection rule to a dataset. Static textbook examples do not always provide an opportunity to change observations, compare detection criteria, receive immediate feedback, or review progress. A compact learning tool is therefore useful if it allows the learner to move from intuitive recognition to formal statistical reasoning.

2.2 Aim

The aim is to design and implement an interactive simulator that helps learners practise identifying categorical oddities, unusual numerical observations, and statistical outliers using established introductory methods.

2.3 Objectives

Objective	Corresponding capability
Introduce the concept of an unusual observation.	Category, numerical and statistical question types.
Provide progressive practice.	Four game modes and four difficulty levels.
Demonstrate formal detection rules.	IQR fences and Z-score threshold.
Make numerical reasoning visible.	Interactive dot plot and displayed descriptive statistics.
Support learning through feedback.	Hints, correctness feedback and explanations.

Encourage repeated practice.	Timed rounds, points, streaks and session summaries.
Retain local progress.	Browser localStorage-based statistics and recent history.

2.4 Intended Users

The intended users are students and beginners studying data science, statistics, or introductory data analysis. The simulator is designed for practice and concept reinforcement; it is not intended to replace a complete statistics course or domain-specific professional analysis.

2.5 Expected Learning Outcomes

- Distinguish a conceptual category mismatch from a statistical outlier.
- Explain the meaning of Q1, Q3, IQR and the 1.5×IQR fences.
- Calculate or interpret a Z-score relative to a dataset mean and standard deviation.
- Recognise that different detection rules can classify the same observation differently.
- Interpret a flagged observation using the surrounding dataset and its domain context.

CHAPTER 3: THEORETICAL BACKGROUND

3.1 Definition and Interpretation of Outliers

An outlier is an observation that differs markedly from the general pattern of the data. The term is descriptive rather than causal: it indicates unusualness under a chosen reference model or rule, not the reason why the observation occurred. An observation flagged as an outlier should be investigated rather than automatically deleted.

3.2 Types of Exercises in the Simulator

The application deliberately distinguishes three kinds of tasks. Category Clash asks which item violates a conceptual grouping—for example, identifying a vegetable among fruits. This is a semantic classification exercise and does not calculate statistical distance. Number Detective asks the learner to identify a numerical value that is separated from a cluster or violates an

evident sequence pattern. Data Scientist Challenge applies a formal statistical rule to a dataset and target value. These task types build from intuitive pattern recognition toward rule-based analysis.

3.3 Interquartile Range Method

The interquartile range measures the spread of the middle half of an ordered dataset. Q1 is the 25th percentile and Q3 is the 75th percentile. The IQR is the difference between them:

$$\mathbf{IQR = Q3 - Q1}$$

The conventional 1.5×IQR fences are:

$$\mathbf{Lower\ Fence = Q1 - 1.5 \times IQR} \quad | \quad \mathbf{Upper\ Fence = Q3 + 1.5 \times IQR}$$

A value below the lower fence or above the upper fence is flagged by this rule. Values on the fences are not flagged because the implementation uses strict less-than and greater-than comparisons. The fences are screening limits, not proof that an observation is invalid.

3.4 Z-score Method

A Z-score expresses the distance of an observation from the mean in standard-deviation units:

$$\mathbf{Z = (x - \mu) / \sigma}$$

Here x is the observation, μ is the dataset mean, and σ is the standard deviation used by the implementation. The simulator uses $|Z| > 2.0$ as its default decision threshold. This is an educational threshold and should not be interpreted as a universal rule for every distribution. In the code, standard deviation is calculated using division by n, i.e. the population form.

3.5 Quartile Calculation in the Implementation

The statistics module sorts the finite observations and computes each percentile using linear interpolation. For percentile p and sorted array length n, the zero-based index is $h = (n - 1)p$. If h is not an integer, the implementation interpolates between the values at floor(h) and ceil(h). This is one recognised percentile convention; other statistical packages may use different quartile conventions, so small differences can occur when comparing results across tools.

3.6 Worked Example

Consider the dataset [8, 9, 10, 11, 75]. Using the implementation's linear-interpolation percentile calculation, Q1 = 9, Q3 = 11, and IQR = 2. The upper fence is 11 + (1.5 × 2) = 14, while the lower fence is 9 − 3 = 6. Since 75 is greater than 14, it is flagged by the IQR rule.

Quantity	Value
Q1	9
Q3	11
IQR	2
Lower fence	6
Upper fence	14
Target observation	75
IQR decision	Outlier

3.7 Limitations of Rule-Based Detection

The IQR rule is resistant to extreme values in the sense that quartiles are less affected by extremes than the mean and standard deviation, but it may not identify every unusual observation in every distribution. A Z-score depends on the mean and standard deviation and can be influenced by extreme values. A fixed threshold such as 2.0 is a heuristic, not a universal statistical guarantee. The result should be interpreted with distribution shape, sample size, measurement process and domain knowledge in mind.

3.8 Synthetic Data

The project includes a synthetic dataset generator. It creates between 12 and 16 observations around a category-specific mean and spread using a Box–Muller normal random-number transformation. Depending on the selected option, it generates a target above or below the IQR fence, or a target within the central quartile interval. The available categories include exam scores, temperature, latency, pricing and step counts. Generated values are practice examples and are not empirical measurements.

CHAPTER 4: SYSTEM ANALYSIS AND REQUIREMENTS

4.1 Existing Learning Approach and Identified Need

Conventional learning resources commonly explain outliers through definitions, formulae, worked examples and static charts. Quiz applications may add scoring but often do not expose the statistical calculation behind a decision. Outlier Hunt addresses this learning need by combining conceptual questions, numerical challenges, a formula-driven statistical mode, an explanatory feedback loop and a local performance record.

4.2 Functional Requirements

ID	Requirement
FR-01	The application shall present four selectable game modes.
FR-02	The learner shall be able to select one of four difficulty levels.
FR-03	A session shall present five questions selected from the question bank.
FR-04	The application shall display a countdown timer during a question.
FR-05	The learner shall be able to request a hint and skip a question.
FR-06	The system shall evaluate categorical choices and statistical yes/no decisions.
FR-07	The statistical mode shall accept an editable dataset and target observation.
FR-08	The learner shall be able to choose IQR or Z-score detection.
FR-09	The application shall display descriptive statistics and a data chart.
FR-10	The application shall calculate score, accuracy, streak and elapsed time.
FR-11	The system shall store performance statistics

	locally in the browser.
FR-12	The learner shall be able to review recent performance and clear local history.

4.3 Non-functional Requirements

Quality attribute	Project interpretation
Usability	A visually structured interface with clear mode selection, controls and feedback.
Responsiveness	The layout uses responsive utility classes for different viewport widths.
Performance	Statistical calculations are performed locally on small in-memory arrays.
Reliability	Input values are filtered to finite numbers; insufficient datasets return no statistics.
Privacy / data handling	Performance records are stored in the browser rather than sent to a project database.
Maintainability	The source is separated into components, data definitions, statistical utilities and storage utilities.

4.4 Constraints and Assumptions

The core learning workflow is client-side. It assumes a modern browser with JavaScript enabled and localStorage available. Local records are specific to the browser profile and device and may be removed by clearing browser data. The project does not demonstrate centralised synchronisation, authentication, or a remote database for player progress.

4.5 Use Case Summary

Actor	Use case	Outcome
Learner	Select mode and difficulty	A game session is configured.
Learner	Answer, request hint or skip	The current question is evaluated or recorded as

		skipped.
Learner	Edit dataset and target	The statistical summary and classification update.
Learner	Choose IQR or Z-score	The selected decision rule is applied.
Learner	Review statistics	The locally stored performance summary is displayed.
Learner	Clear history	Saved local performance data is removed.

CHAPTER 5: SYSTEM DESIGN AND ARCHITECTURE

5.1 Architectural Overview

The application follows a component-based frontend architecture. React components manage the visible screens and interaction state. Statistical calculations and synthetic-data generation are isolated in a utility module. The question bank is held as structured TypeScript data. A separate storage module loads and updates performance records through browser localStorage. The application repository also includes a TanStack Start/Router structure and a Vite build configuration.

The architecture can be represented as four cooperating layers: (1) presentation components, (2) game-session and interaction logic, (3) question and statistical domain modules, and (4) browser persistence. The statistical decision is computed in the browser; the evidence reviewed does not show a remote statistical-processing API or a project database.

Module / component	**Responsibility**
ModeSelector	Mode and difficulty selection.
GameplayScreen	Question flow, timer, scoring, hints, answer capture and statistical challenge state.
DataChart	Visual representation of sorted data, IQR region, fences, mean and target observation.
DatasetControls	Dataset, target, detection rule and shuffle controls.

ResultsScreen	Final score, accuracy, streak, elapsed time and answer-wise review.
Stats / statistics utility	Mean, median, standard deviation, quartiles, IQR fences, Z-score and synthetic data.
Question bank	48 structured questions across four modes.
Storage utility	Load, save, update and clear local performance records.

5.2 Data Flow

1. The learner selects a mode and difficulty.
2. The application filters the question bank and selects a five-question session.
3. For each question, the gameplay component initialises the timer and question-specific state.
4. The learner submits a categorical choice or a binary outlier judgement.
5. The application evaluates the answer, calculates points and updates streak and answer history.
6. At the end of the session, the results screen summarises the recorded answers.
7. The session record is incorporated into browser-local statistics and recent history.

5.3 Question Bank Design

The question bank contains 48 question records: 12 each for Quick Hunt, Category Clash, Number Detective and Data Scientist Challenge. The records include identifiers, mode, difficulty, category, type, prompt, hint, explanation and supporting pattern information. Categorical questions also include answer options and a boolean indicator identifying the intended odd item. The selection function first filters by the requested mode and difficulty; if fewer than the requested number are available, it widens the pool to all questions in that mode. The selected pool is shuffled and the first five records are used.

5.4 Game State and Scoring

The gameplay component maintains the current question index, score, current and best streak, submitted answers, elapsed time, selected option, answer status, hint usage and countdown.

Difficulty sets the base time: 45 seconds for Easy, 35 for Medium, 25 for Hard and 20 for Expert. The points multiplier is respectively 1.0, 1.25, 1.5 and 2.0.

For a correct response, the code uses a base of 100 points, a time bonus up to 50 points, a streak bonus of 20 points for each streak step after the first, and a 30-point hint penalty. The result is multiplied by the difficulty multiplier and bounded below by 20 points. Incorrect answers, skipped questions and timeouts earn zero points and reset the current streak.

5.5 Statistical Data Flow

The editable dataset text is split on whitespace, commas or semicolons; tokens are converted to numbers and non-finite values are excluded. The statistics function requires at least three valid observations. It sorts the values, calculates the mean and population standard deviation, obtains Q1, median and Q3 through linear interpolation, and derives the IQR fences. The target observation is then tested against the selected rule. The chart uses the selected rule to display the corresponding fence interval.

5.6 Persistence Design

The storage module uses the key outlier_hunt_user_stats in localStorage. It maintains total games played, overall high score, best streak, total questions answered, correct answers, mode-specific high scores, difficulty completion counts and a history of recent game records. The history is limited to the latest 50 records. The sound preference is stored separately under outlier_hunt_muted.

5.7 Architecture Diagram (Textual Representation)

Learner → React user interface → Game/session state → Question bank and statistics utility → Answer evaluation and results → localStorage performance record

For the Data Scientist Challenge, the statistical path is: dataset input → numeric parsing → sorted observations → descriptive statistics and quartiles → IQR/Z-score rule → chart and verdict → feedback.

CHAPTER 6: IMPLEMENTATION DETAILS

6.1 Technology Stack

Technology	Role evidenced by project configuration
React 19	Component-based user interface and stateful interaction.
TypeScript	Typed question, game, answer and statistics structures.
Vite	Development server and production build tooling.
TanStack Start / Router	Application routing and application shell.
Tailwind CSS	Utility-based interface styling; Tailwind Vite integration is configured.
Framer Motion	Animation library listed as a runtime dependency.
Lucide React	Interface icons.
Vitest	Automated unit-test framework.
Browser localStorage	Local persistence for player statistics and sound preference.
Canvas Confetti	Celebratory effect on high-accuracy results.

The project configuration also includes numerous supporting UI packages. The table lists the principal technologies relevant to the documented architecture rather than every installed package.

6.2 Statistical Utility Implementation

The calculateStats function first removes non-finite values and returns null if fewer than three observations remain. It creates a sorted copy, computes the mean as the arithmetic average, and computes variance by summing squared deviations from the mean and dividing by n. The square root of this variance is the population standard deviation. Percentiles are calculated using the position $(n - 1)p$ and linear interpolation between adjacent sorted values.

The isValueOutlier function applies strict fence comparisons for IQR. For Z-score, it calculates the absolute standardised distance and compares it with the supplied threshold, which defaults to 2.0. If the standard deviation is zero, calculateZScore returns zero, preventing division by zero.

6.3 Synthetic Dataset Generation

The generator selects a base mean, spread, unit and description according to the requested category. It creates 12–16 values using a Box–Muller transform and rounds the generated observations. When an outlier is requested, the target is placed beyond an IQR fence by a category-scaled distance. Otherwise, the target is selected within the interval from Q1 to Q3. Because the values are generated randomly, the precise dataset varies between runs.

6.4 Feedback and Audio

The gameplay logic records the selected answer, correctness, time spent, hint use, points earned and explanation. Correct and incorrect responses trigger different sound methods when sound is enabled. The sound manager uses the Web Audio API to generate short oscillator tones and stores the mute preference locally. The default state is muted.

6.5 Local Statistics

After a completed game, the storage utility updates aggregate counts and high scores and prepends the new game record to the history. The latest 50 records are retained. This design provides a simple personal progress view without requiring a server account. It also means that progress is not automatically shared across browsers or devices.

6.6 Error and Input Handling

The statistics module excludes invalid numeric tokens and requires at least three observations. The local-storage loader uses a try/catch block and merges stored values with default structures. These safeguards improve resilience for malformed input or invalid stored JSON, although they do not constitute comprehensive validation for every possible user input or browser failure.

CHAPTER 7: USER INTERFACE AND SCREENSHOTS

7.1 Mode and Difficulty Selection

The mode-selection screen presents four cards with descriptions and examples. The learner can select Quick Hunt, Category Clash, Number Detective or Data Scientist Challenge and choose a difficulty level. The interface also shows the best score associated with each mode.

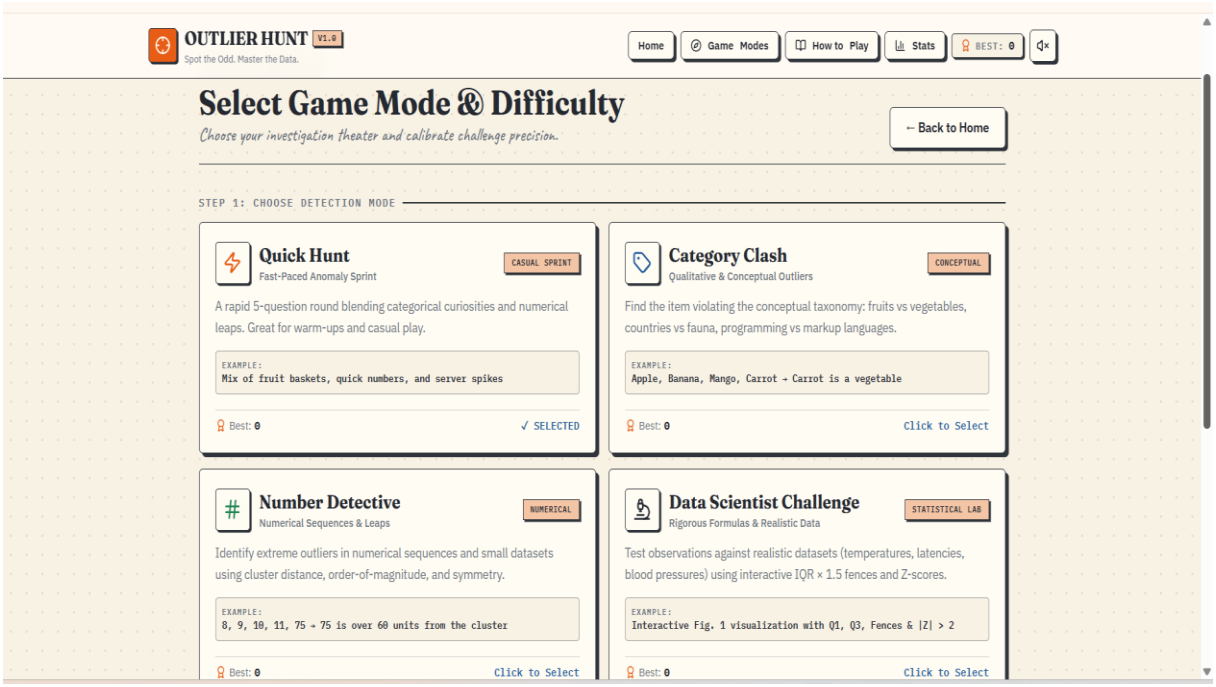


Figure 4.1. Game-mode and difficulty selection interface.

7.2 Statistical Investigation Interface

The supplied Data Scientist Challenge screenshot shows a numerical dataset, a target observation of 9.4 microseconds, an IQR-based chart, and the descriptive-statistics strip. The visible chart marks the central IQR region, the 1.5×IQR fences, the mean and the target observation. The controls panel exposes the raw dataset, target value and the choice between IQR × 1.5 and |Z-score| > 2.0. The verdict area asks the learner to decide whether the target is an outlier under the selected rule.

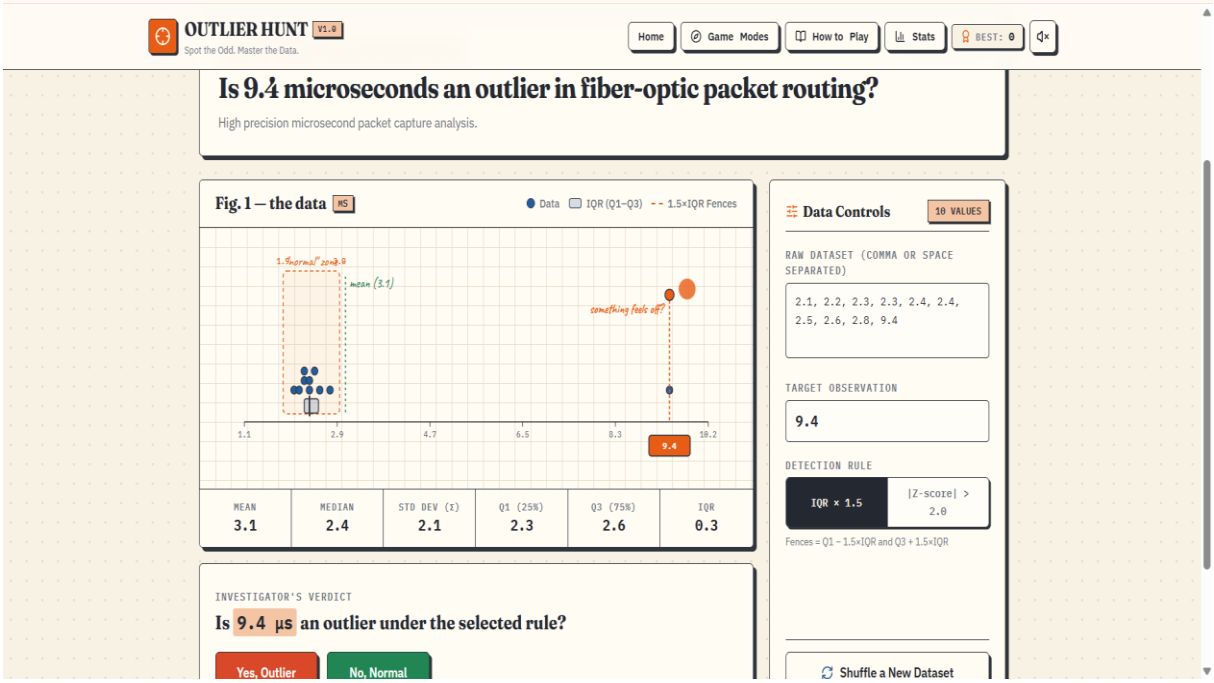


Figure 5.1. Data Scientist Challenge: dataset controls, descriptive statistics, IQR fences and target observation.

7.3 Personal Statistics Interface

The Personal Statistics panel displays total games, high score, best streak and accuracy, followed by mode-specific high scores and recent investigations. The screenshot demonstrates that the interface labels the information as a local dossier. This is consistent with the localStorage-based implementation and should not be interpreted as a cloud-synchronised user profile.

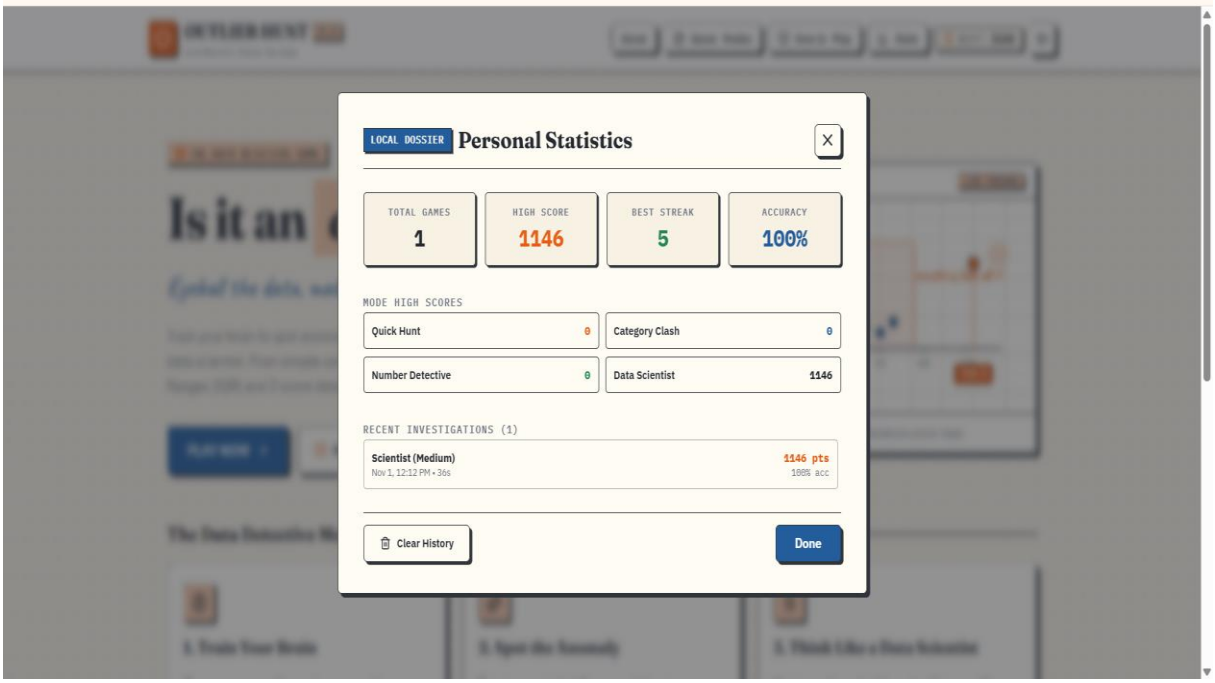


Figure 6.1. Personal Statistics panel showing browser-stored game performance and recent investigations.

7.4 Interface Design Considerations

The application uses a consistent visual language with card-based panels, high-contrast labels, a restrained colour palette and typography that distinguishes headings, explanatory text and numerical values. The game interface places time, score and streak information in a status area so that the learner can monitor progress while answering. The statistical view prioritises the dataset and its reference limits, making the selected rule visible alongside the target observation.

CHAPTER 8: TESTING AND VERIFICATION

8.1 Testing Approach

The repository includes a Vitest test file covering statistical calculations, IQR classification, Z-score calculation, question-bank structure and session question selection. The source-level tests provide evidence that these cases have been specified as automated checks. This report does not claim that the test suite was executed during report preparation; execution results should be recorded only after running the project tests in the development environment.

8.2 Test Cases Evidenced in the Repository

Test case	Input / condition	Expected assertion in source
Descriptive statistics	[10, 20, 30, 40, 50]	Mean 30; median 30; Q1 20; Q3 40; IQR 20; fences −10 and 70.
IQR outlier classification	[8, 9, 10, 11, 75]	75 is flagged; 10 is not flagged.
Z-score calculation	x=125, mean=100, SD=10	Z-score is 2.5.
Boundary example	x=80, mean=100, SD=10	Z-score is −2.0.
Question bank size	QUESTION_BANK	At least 40 questions.
Question structure	Each question	ID, prompt, explanation, hint and pattern fields are present; categorical items have at least three options and exactly one marked outlier.
Session selection	Selected modes and difficulty	Five questions are returned for tested mode/difficulty combinations.

8.3 Additional Manual Verification Recommended

- Enter a dataset with fewer than three valid numbers and confirm that statistics are not displayed as if valid.
- Enter malformed tokens alongside valid numbers and verify that only finite numeric values are processed.
- Test a target exactly equal to either IQR fence; strict comparisons should classify it as not outside the fence.
- Compare IQR and Z-score decisions for a dataset with a strong extreme value.
- Test timer expiry, skip, hint penalty, streak reset and end-of-session navigation.
- Clear local history and refresh the page to confirm the intended persistence behaviour.
- Repeat a game in a different browser profile to verify that local progress is not synchronised.

8.4 Interpretation of the Evidence

The unit tests establish intended mathematical and structural assertions in source code, while the screenshots demonstrate selected user-interface states. Neither source-level tests nor static screenshots alone establish usability, learning gain, statistical accuracy across all possible distributions, or production reliability. A formal evaluation would require executed test logs and, for learning effectiveness, a documented user study with a defined protocol.

CHAPTER 9: NOVELTY, EVALUATION AND LIMITATIONS

9.1 What Has Been Built

The project delivers a browser-based educational simulator that combines conceptual odd-one-out tasks, numerical pattern recognition and formula-based statistical outlier decisions. It provides a four-mode structure, four difficulty levels, a 48-question bank, interactive statistical controls, visual feedback, scoring, hints and locally stored performance summaries.

9.2 How It Has Been Built

The application is organised around React components and typed data structures. The game flow is managed through component state and effects. The statistical logic is isolated in a reusable TypeScript utility, and the question bank is separated from presentation code. The Data Scientist Challenge connects user-editable data to descriptive-statistics calculations, a selectable decision rule, a chart and a binary verdict. Browser localStorage retains the personal performance record.

9.3 Technical Significance

The project makes the decision process more inspectable than a static answer-only quiz. The learner can see the data distribution, central quartile interval, fences, mean, target observation and computed summary values before making a decision. The separation of conceptual, numerical and statistical tasks also highlights that not every unusual item is a statistical outlier. This is a meaningful instructional design choice because it prevents a semantic category mismatch from being confused with a numerical threshold test.

9.4 Evidence-Based Comparison

Approach	Typical learning interaction	Outlier Hunt distinction
Static textbook example	Read a definition, formula and worked dataset.	Adds interactive question flow, immediate feedback and repeatable practice.
General quiz application	Select an answer and receive a score.	Adds a dedicated statistical mode with editable data and visible IQR/Z-score criteria.
Standalone statistical calculator	Enter data and receive numerical summaries.	Places calculations inside a guided learning activity with a target observation and explanation.
Concept-only odd-one-out game	Identify an item that does not fit a group.	Explicitly separates conceptual mismatch from numerical and statistical detection.

This comparison is a design-level comparison, not the result of a controlled benchmark against named commercial products. The project does not claim algorithmic novelty: IQR fences, Z-scores, percentile interpolation and Box–Muller sampling are established techniques. The defensible contribution is their integration into a single beginner-oriented game-based learning workflow.

9.5 Limitations

- The question bank is finite and predefined; the project does not demonstrate automatic generation of new explanatory questions.
- The statistical rules are introductory heuristics and do not model every distribution or domain-specific anomaly problem.
- The default Z-score threshold of 2.0 may not be suitable for all datasets.
- The application stores progress locally, so records are browser/device-specific and can be cleared by the user or browser.
- No empirical study of learning improvement or usability is included in the supplied evidence.

- No multi-user authentication, remote database, or centralised analytics service is evidenced in the reviewed source.
- The simulator is not a substitute for expert domain investigation of flagged observations.

9.6 Future Scope

- Add a structured tutorial that explains each statistical calculation step by step.
- Provide additional robust methods such as modified Z-scores based on median absolute deviation, with clear explanations of their assumptions.
- Add distribution-shape examples and demonstrate how skewness and extreme values affect detection rules.
- Provide exportable session summaries for academic practice records.
- Introduce optional user accounts and secure synchronisation only if there is a clear need for cross-device progress.
- Add accessibility improvements, keyboard navigation and a systematic responsive-interface evaluation.
- Conduct a small controlled learning evaluation using pre-test and post-test questions and report the results transparently.

CHAPTER 10: CONCLUSION AND FUTURE SCOPE

Outlier Hunt demonstrates how an introductory statistical concept can be presented through an interactive learning environment. The simulator moves from conceptual odd-one-out recognition to numerical pattern identification and formal IQR/Z-score decisions. The implementation combines a structured question bank, timed game sessions, hints, feedback, visual statistical summaries and browser-local progress records.

The source review confirms that the statistical calculations are implemented locally in TypeScript and that the statistical mode exposes the selected detection rule and target observation to the learner. The application uses established methods rather than proposing a new outlier-detection algorithm. Its principal academic value is the integration of these methods into a guided practice experience that makes the distinction between conceptual unusualness and statistical unusualness explicit.

Further work should focus on richer statistical explanations, broader datasets, accessibility, and empirical evaluation of learning outcomes. Any future claims about effectiveness should be supported by a documented evaluation rather than inferred from the presence of game mechanics.

REFERENCES

[1] NIST/SEMATECH, e-Handbook of Statistical Methods, “Exploratory Data Analysis” and related sections on graphical and quantitative data analysis. National Institute of Standards and Technology. https://www.itl.nist.gov/div898/handbook/

[2] Tukey, J. W. (1977). Exploratory Data Analysis. Addison-Wesley.

[3] Montgomery, D. C., & Runger, G. C. (2018). Applied Statistics and Probability for Engineers (7th ed.). Wiley.

[4] React Team. React Documentation. https://react.dev/

[5] Microsoft. TypeScript Handbook. https://www.typescriptlang.org/docs/handbook/intro.html

[6] Vite Team. Vite Guide. https://vite.dev/guide/

[7] Project source repository: prerna12github/Outlier-Detection-Game. https://github.com/prerna12github/Outlier-Detection-Game (accessed 1 November 2026).

[8] Outlier Hunt project presentation, revised version supplied for this report.

[9] Outlier Hunt application screenshots supplied by the project team; live application: https://outlier-hunt.prernabuilds.me.

APPENDIX A: IMPORTANT FORMULAE AND ALGORITHMS

A.1 IQR Rule

1. Sort the valid observations.
2. Calculate Q1 and Q3 using the selected percentile convention.
3. Calculate IQR = Q3 − Q1.

4. Calculate lower = Q1 − 1.5 × IQR and upper = Q3 + 1.5 × IQR.
5. Flag x if x < lower or x > upper.

A.2 Z-score Rule

1. Calculate the dataset mean μ.
2. Calculate standard deviation σ.
3. Calculate Z = (x − μ) / σ.
4. Flag x if |Z| > 2.0. If σ = 0, the implementation returns a Z-score of zero.

A.3 Session Selection

1. Filter the question bank by requested mode and difficulty.
2. If the filtered pool contains fewer than five questions, replace it with all questions for the selected mode.
3. Shuffle the selected pool.
4. Return the first five questions.

APPENDIX B: PROJECT LINKS

Live application: https://outlier-hunt.prernabuilds.me

Source repository: https://github.com/prerna12github/Outlier-Detection-Game